Confluent Platform SQL Deployment via Flink SQL Client — Timer-Driven PTF UDFs

> This example deploys the **Session Timeout Detector** and **Per-Event Follow-Up** PTF UDFs (source in examples/ptf_udf_timer_driven/java/) by submitting SQL statements through the **Flink SQL Client** running directly on the JobManager pod. Both UDFs are packaged in a single JAR.

Table of Contents

- 1.0 Overview
  - 1.1 How this differs from the other deployment paths
- 2.0 How it works
  - 2.1 JAR delivery
  - 2.2 Kafka connector
  - 2.3 Statement flow
- 3.0 Prerequisites
- 4.0 How to run
  - 4.1 Deploy
  - 4.2 Monitor
  - 4.3 Tear down
- 5.0 Resources

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
Where it runs	Confluent Platform on Minikube	Confluent Cloud
How SQL is submitted	`sql-client.sh -f` on the JobManager pod	`confluent_flink_statement` Terraform resources
UDF JAR delivery	`kubectl exec` to Flink pods	`confluent_flink_artifact` (uploaded to CC)
Entry point	`make deploy-cp-ptf-udf-timer-driven`	`make deploy-cc-ptf-udf-timer-driven`
Requires code compilation	Yes (Java + Gradle for UDF JAR)	Yes (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Flink session cluster	Managed by Terraform state

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
Same codebase for both?	Yes (**same Java UDF code**, different Terraform vs SQL Client for deployment)	Yes (**same Java UDF code**, different Terraform vs SQL Client for deployment)

2.0 How it works

2.1 JAR delivery

On Confluent Platform there is no artifact store — the JAR must be physically present on the Flink pods.

The deploy script copies the fat JAR (built from examples/ptf_udf_timer_driven/java/) to `/opt/flink/usrlib/session-timeout-detector.jar` on every JobManager and TaskManager pod using `kubectl exec`. Both `CREATE FUNCTION ... USING JAR` statements reference this pod-local path, registering `session_timeout_detector` and `per_event_follow_up` from the same JAR.

2.2 Kafka connector

The CP Flink base image does not include the Kafka SQL connector. The FlinkDeployment uses two **init containers** to assemble `/opt/flink/lib/` at pod startup:

1. **copy-stock-lib** — copies the stock Flink lib JARs from the image into a shared `emptyDir` volume
2. **fetch-kafka-connector** — downloads `flink-sql-connector-kafka` from Maven Central into the same volume

The main Flink container then mounts this volume at `/opt/flink/lib/`, so the Kafka connector is on the classpath when the JVM starts.

2.3 Statement flow

The script pre-creates Kafka topics, then executes all SQL in a single `sql-client.sh -f` session on the JobManager pod:

```
┌──────────────────────────────────────────────────────────────────┐
│  Pre-step: kafka-topics --create (4 topics)                      │
│                                                                  │
│  ── Named timer pipeline (SessionTimeoutDetector) ──             │
│  Step 1:  DROP TABLE IF EXISTS user_activity            → OK     │
│  Step 2:  CREATE TABLE user_activity (... WITH kafka)   → OK     │
│  Step 3:  INSERT INTO user_activity VALUES (sample data) → submitted │
│  Step 4:  DROP TABLE IF EXISTS timeout_events           → OK     │
│  Step 5:  CREATE TABLE timeout_events (... WITH kafka)  → OK     │
│  Step 6:  CREATE FUNCTION session_timeout_detector      → OK     │
│  Step 7:  INSERT INTO timeout_events                    → submitted │
│                                                                  │
│  ── Unnamed timer pipeline (PerEventFollowUp) ──                 │
│  Step 8:  DROP TABLE IF EXISTS user_actions             → OK     │
│  Step 9:  CREATE TABLE user_actions (... WITH kafka)    → OK     │
```

```
│  Step 10: INSERT INTO user_actions VALUES (sample data) → submitted  │
│  Step 11: DROP TABLE IF EXISTS follow_up_events         → OK         │
│  Step 12: CREATE TABLE follow_up_events (... WITH kafka)→ OK         │
│  Step 13: CREATE FUNCTION per_event_follow_up           → OK         │
│  Step 14: INSERT INTO follow_up_events                  → submitted  │
└──────────────────────────────────────────────────────────────────────┘
```

Steps 7 and 14 are **long-running streaming jobs**. They run continuously, reading from their respective source topics and writing output to sink topics.

3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get, and Docker Desktop running
- Java 17+
- Confluent Platform and Flink stack already deployed:

```
make install-prereqs       # installs tooling (docker, kubectl, minikube, helm, gradle)
make cp-up                 # Minikube → CFK Operator → Kafka + SR + Connect + C3 + Kafka UI
make cp-watch              # watch pods come up (Ctrl+C when all Running)
make flink-up              # cert-manager → Flink Operator → CMF → Flink session cluster
make flink-status          # verify Flink pods are Running
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR, copies it to the Flink pods, and executes all SQL:

```
make deploy-cp-ptf-udf-timer-driven
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF fat JAR from `examples/ptf_udf_timer_driven/java/`
2	`kubectl exec` — copies the JAR to all JobManager and TaskManager pods

Step	What it does
3	`kafka-topics --create` — pre-creates Kafka topics (`user_activity`, `timeout_events`, `user_actions`, `follow_up_events`)
4	`sql-client.sh -f` — executes all SQL statements for both UDF pipelines in a single session on the JobManager pod

4.2 Monitor

Open the Flink Dashboard to see the running jobs:

```
make flink-ui              # opens http://localhost:8081
```

4.3 Tear down

To stop the running jobs and drop all tables and functions:

```
make teardown-cp-ptf-udf-timer-driven
```

This cancels any running Flink jobs via the Flink REST API, then submits `DROP FUNCTION` and `DROP TABLE` statements for both UDFs.

5.0 Resources

- Flink SQL Client
- Flink Kafka Connector
- Process Table Functions (PTFs)
- Create a User-Defined Function (Confluent Cloud)